

A PROJECT REPORT ON

BLOCKCHAIN-BASED ACADEMIC CERTIFICATE VERIFICATION SYSTEM

Domain: Blockchain / Security

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 Software Specification	
V	SYSTEM ANALYSIS	10
	5.1 Feasibility Study	

	5.2 About the Software	
VI	SYSTEM DESIGN	13
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	22
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	29
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	30

CHAPTER I

INTRODUCTION

Academic certificates such as degree certificates, mark sheets and course completion certificates are essential documents for higher studies and employment. Unfortunately, fake and forged certificates have become common because modern printers and editing software make it easy to produce documents that look genuine. Cases of fake degrees being used for jobs, visas and admissions are regularly reported.

At present, an employer who wants to verify a certificate must contact the issuing university by letter or e-mail, often pay a fee, and wait for several weeks. Central databases of certificates exist in some places, but they can be tampered with by insiders or attackers, and they depend on a single organisation being available.

Blockchain technology offers a tamper-proof, decentralised ledger in which records, once written, cannot be modified. By storing the cryptographic hash of each certificate on a blockchain through a smart contract, any verifier can instantly check whether a given document is exactly the one that was issued. This project, CertChain, enables colleges to issue certificates whose SHA-256 hashes are stored on an Ethereum test network and allows employers to verify any certificate by uploading it or scanning its QR code.

LIMITATIONS

1. Manual verification takes weeks and involves cost.
2. Centralised databases can be altered by insiders.
3. Students cannot easily prove their credentials across borders.
4. Forged certificates are difficult to detect visually.

1.1 History of Domain

The concept of a cryptographically secured chain of blocks was described by Haber and Stornetta in 1991 for time-stamping digital documents. In 2008 Satoshi

Nakamoto published the Bitcoin white paper, which combined a hash-linked chain with proof-of-work consensus to create the first decentralised digital currency.

Ethereum, proposed by Vitalik Buterin and launched in 2015, introduced smart contracts that allow arbitrary programs to run on the blockchain. Since then, blockchain has been applied to supply chains, land records and education. MIT Media Lab released Blockcerts in 2016 as an open standard for blockchain-based academic credentials, and several universities now issue digital diplomas anchored on blockchains.

1.2 Overview of the Project

In CertChain, an authorised institution logs in and uploads a certificate PDF along with student details. The system computes the SHA-256 hash of the file, uploads the file to IPFS, and calls the `issueCertificate` function of a Solidity smart contract deployed on the Sepolia test network. The contract stores the hash, the issuer address, the IPFS CID and the timestamp. A QR code containing the certificate ID and verification link is generated and printed on the certificate.

A verifier, such as an employer, either scans the QR code or uploads the certificate file on the verification page. The system recomputes the hash and queries the smart contract. If the hash exists and has not been revoked, the result is Valid; if the certificate ID exists but the hash differs, the result is Tampered; otherwise it is Not Found. Every verification is recorded in an audit log.

1.3 Motivation

Recruiters and universities spend considerable time and money verifying certificates, while genuine students suffer delays and forgers take advantage of the gap.

- **Motivation:** To make certificate verification instant, free and tamper-proof for anyone in the world.
- **Outcome:** Elimination of forged certificates, faster recruitment and portable digital credentials for students.

1.4 Aim of the Project

The project is entitled as “**BLOCKCHAIN-BASED ACADEMIC CERTIFICATE VERIFICATION SYSTEM**”. The aim of the project is to develop a decentralised certificate issuance and verification system in which the integrity of every certificate is guaranteed by a blockchain smart contract and can be verified instantly through file upload or QR code scan.

1.5 Objectives

- To write and deploy a Solidity smart contract for issuing, verifying and revoking certificates.
- To compute SHA-256 hashes of certificate files and store them on chain.
- To store certificate files on IPFS.
- To generate QR codes for quick verification.
- To maintain an audit log of all verifications.

1.6 Literature Review

Nakamoto (2008): Proposed Bitcoin, a peer-to-peer electronic cash system using a proof-of-work chain of blocks to prevent double spending without a trusted third party.

Buterin (2014): The Ethereum white paper described a blockchain with a built-in Turing-complete programming language that allows anyone to write smart contracts and decentralised applications.

Grech and Camilleri (2017): In a European Commission JRC report, analysed the potential of blockchain in education, particularly for self-sovereign academic credentials and the reduction of certificate fraud.

MIT Media Lab - Blockcerts (2016): Released an open standard and reference implementation for creating, issuing and verifying blockchain-based certificates, which anchors a hash of a batch of certificates in a blockchain transaction.

Benet (2014): Proposed IPFS, a content-addressed, peer-to-peer distributed file system in which files are identified by the hash of their content.

NIST FIPS 180-4 (2015): Specifies the Secure Hash Standard including SHA-256, a cryptographic hash function that produces a 256-bit digest for which finding collisions is computationally infeasible.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

In the existing system, universities issue printed certificates with seals and signatures. Verification is done by sending the certificate copy to the university, which checks its internal records and replies by post or e-mail. Some institutions use central online portals, but these depend on a single server and administrator.

DISADVANTAGES

- Verification takes days or weeks.
- Paper certificates are easy to forge.
- Central databases can be tampered with or go offline.
- Verification often requires fees and paperwork.
- No audit trail of verification requests.

2.2 Proposed System

The proposed CertChain system stores a SHA-256 fingerprint of every certificate on the Ethereum blockchain through a smart contract. Since the blockchain is immutable and replicated across thousands of nodes, the stored hash cannot be altered. Verification is instant through QR code or file upload, revocation is supported, and every verification is logged.

ADVANTAGES

- Tamper-proof certificate records.
- Instant verification at no cost to the verifier.
- Any change in the document, even one character, is detected.
- No dependency on a single central server.
- Students carry portable, verifiable credentials.

2.3 Project Schedule

The development of CertChain follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the CertChain system are listed below.

- **Institution registration:** Admin approves institutions and whitelists their wallet addresses in the contract.
- **Certificate issue:** Institution uploads a certificate; the hash is stored on chain.
- **IPFS storage:** Certificate file is stored on IPFS and its CID recorded.
- **QR generation:** QR code with certificate ID and verify URL is generated.
- **Verification:** Verifier uploads file or scans QR; result is Valid, Tampered or Not Found.
- **Revocation:** Issuer can revoke a certificate issued in error.
- **Audit log:** All verifications are logged with time and verifier details.
- **Student view:** Students view and share their certificates.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Security:** Only whitelisted issuer addresses can issue or revoke.

- **Integrity:** SHA-256 ensures any modification is detected.
- **Availability:** Verification works as long as the blockchain is reachable.
- **Performance:** Verification under 3 seconds (read-only call).
- **Cost:** Only hash stored on chain to minimise gas fees.
- **Privacy:** No personal data stored on chain other than hash and ID.

3.3 User Requirements

- Institutions can issue certificates easily from a web page
- Verifiers can check a certificate by upload or QR scan
- Students can share a verification link
- Clear result: Valid, Tampered or Not Found
- History of verifications

3.4 System Requirements

- Ethereum Sepolia test network access (Infura/Alchemy)
- MetaMask wallet for issuers
- Node.js back end and React front end
- IPFS node or pinning service
- MongoDB/PostgreSQL for off-chain metadata and logs

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System	: Intel i5 Processor 2.4 GHz or above
Hard Disk	: 256 GB
RAM	: 8 GB
Monitor	: 15" colour monitor
Scanner / Mobile	: Smartphone camera for QR scanning

4.2 Software Specification

Operating System	: Windows 10 / Ubuntu 22.04
Smart Contract	: Solidity 0.8.x
Blockchain	: Ethereum Sepolia testnet (or Hyperledger Fabric)
Development Tools	: Hardhat, MetaMask
Back End	: Node.js with Express, Ethers.js / Web3.js
Front End	: React JS
Storage	: IPFS (Pinata)
Database	: MongoDB (audit log)

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed CertChain system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

Ethereum, Solidity and Hardhat are widely used and well documented. Test networks provide free test Ether for development. SHA-256 hashing and QR code generation are available in standard libraries. The project is technically feasible.

ECONOMIC FEASIBILITY

All development tools are free. On a public main network, storing a 32-byte hash costs only a small gas fee per certificate, and batching can reduce this further; a permissioned network like Hyperledger has no gas fees. Verification is a free read call. The system is economically feasible.

OPERATIONAL FEASIBILITY

Institutions use a simple web form; verifiers only scan a QR code. No knowledge of blockchain is needed by end users. Hence it is operationally feasible.

5.2 About the Software

ETHEREUM AND SOLIDITY

Ethereum is a decentralised, open-source blockchain with smart contract functionality. A smart contract is a program stored on the blockchain that runs exactly as written; once deployed, its code and stored data cannot be altered by any single party.

Solidity is an object-oriented, statically typed language for writing smart contracts on Ethereum. Contracts can be tested on public test networks such as Sepolia using free

test Ether before deployment to the main network.

- Immutable, tamper-proof records
- Transparent and publicly verifiable
- No central authority required

WEB3.JS / ETHERS.JS

Web3.js and Ethers.js are JavaScript libraries that allow web applications to interact with an Ethereum node over HTTP or WebSocket. They are used to call smart contract functions, send transactions and listen for events.

Combined with a browser wallet such as MetaMask, they let an institution sign transactions securely from the browser.

IPFS (INTERPLANETARY FILE SYSTEM)

IPFS is a peer-to-peer distributed file system that identifies files by their content hash (CID) instead of their location. Any change in the file produces a completely different CID.

Large documents are stored on IPFS and only their CID and hash are stored on the blockchain, which keeps transaction costs low.

NODE.JS AND EXPRESS

Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's V8 engine. It uses an event-driven, non-blocking I/O model that makes it lightweight and efficient for data-intensive real-time applications.

Express.js is a minimal and flexible Node.js web framework that provides a robust set of features for building REST APIs, including routing, middleware and request/response handling. Together they allow both front end and back end to be written in JavaScript.

- Handles many concurrent connections
- npm - the largest package registry

- Ideal for real-time and IoT back ends

REACT JS

React is an open-source JavaScript library for building user interfaces, maintained by Meta and a community of developers. It lets developers build reusable UI components and manages the view layer of web applications efficiently using a virtual DOM.

React applications are composed of components that hold their own state and re-render automatically when data changes. With libraries such as React Router and Axios, a single-page application can communicate with REST APIs and give the user a fast, app-like experience.

- Component based, reusable UI
- Virtual DOM for fast rendering
- Large ecosystem of UI libraries

MONGODB

MongoDB is a source-available, cross-platform, document-oriented NoSQL database. It stores data as flexible JSON-like documents (BSON), which means fields can vary from document to document and the data structure can be changed over time.

MongoDB supports indexing, aggregation pipelines, geospatial queries and time-series collections, which makes it well suited for storing sensor readings, user profiles and application logs.

- Schema-less flexible documents
- Horizontal scaling through sharding
- Powerful aggregation framework

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of CertChain is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

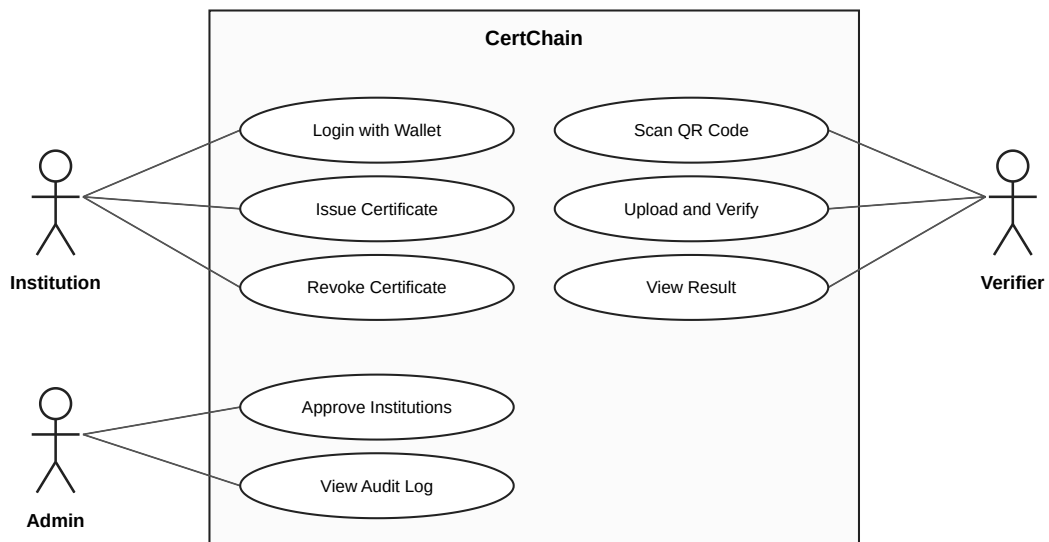


Fig 1: Use case diagram of CertChain

6.2 System Architecture

CertChain uses a hybrid on-chain/off-chain architecture. Only the hash, issuer and status are stored on chain; the file is stored on IPFS and the audit log in MongoDB.

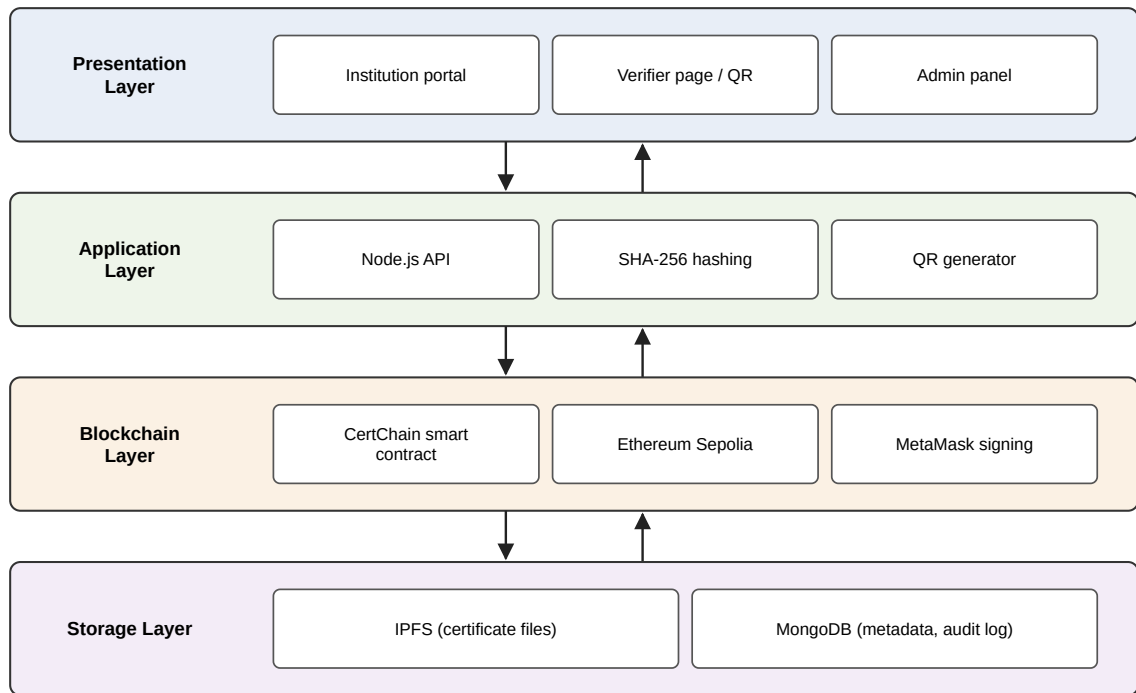


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

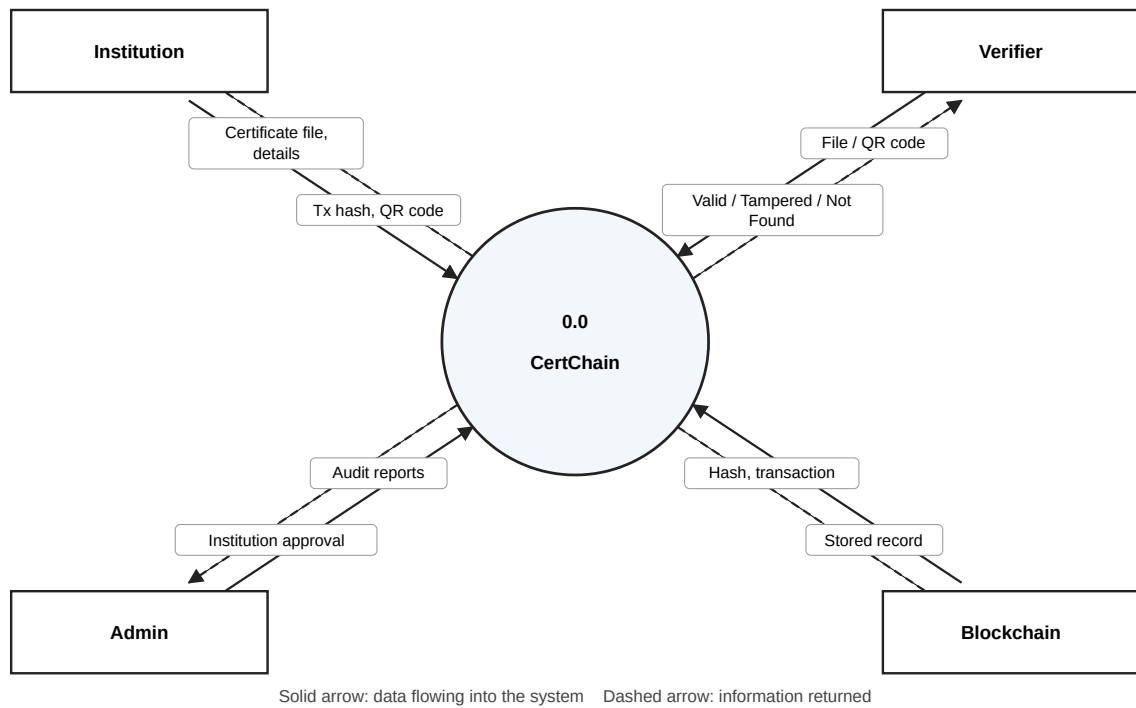


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

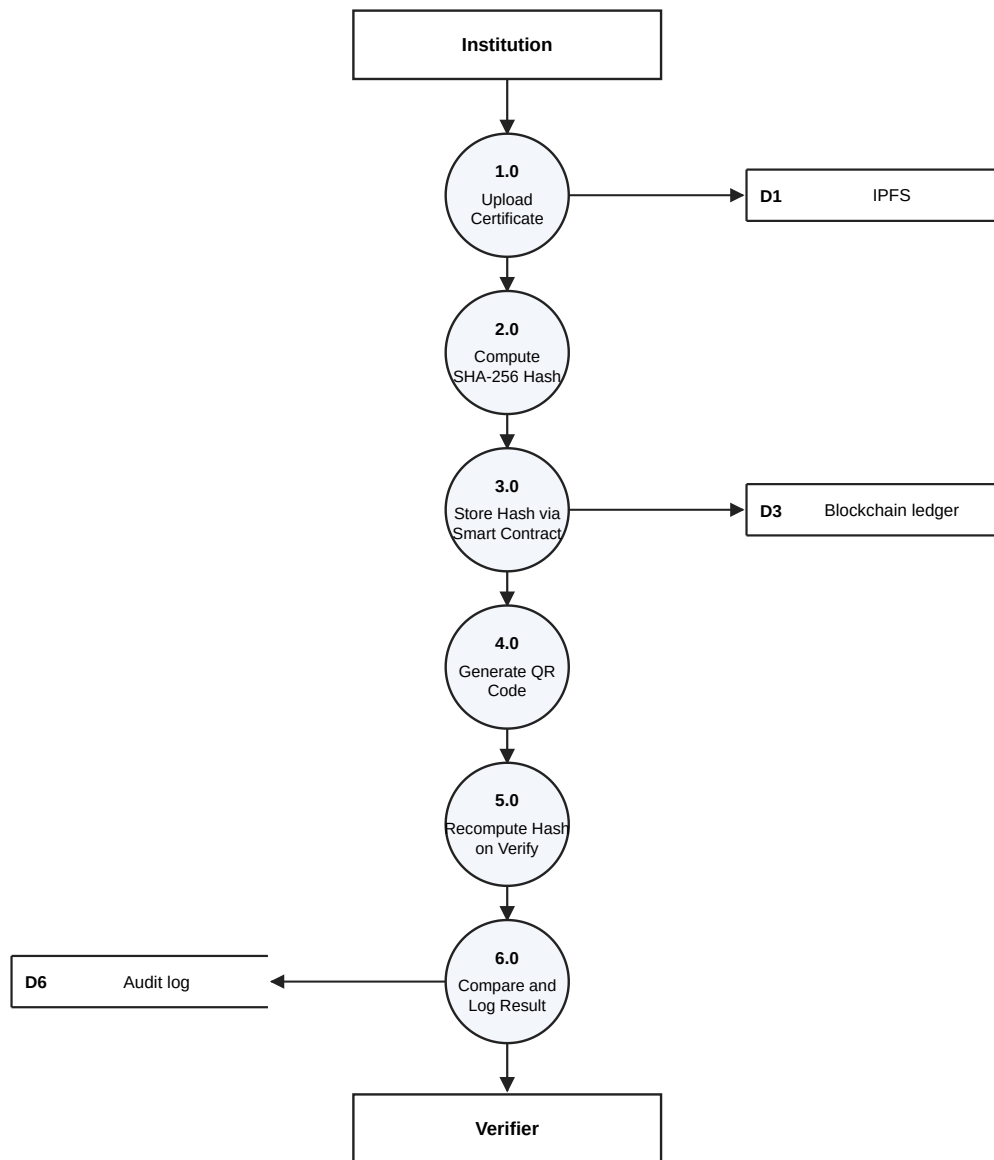


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

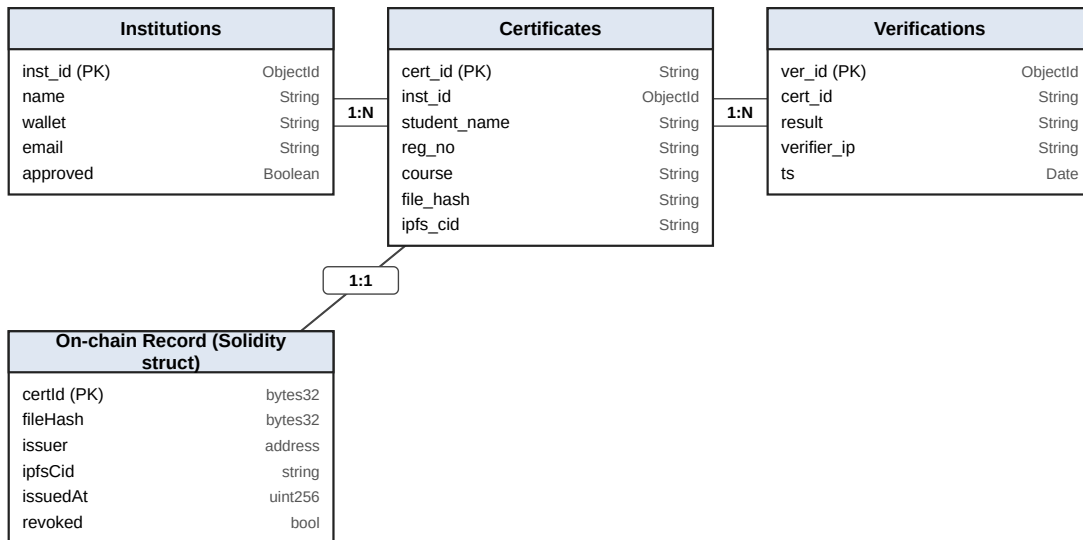


Fig 5: Entity relationship diagram

6.5 Table Design

INSTITUTIONS

FIELD NAME	DATA TYPE	DESCRIPTION
inst_id	ObjectId	Primary key
name	String	Institution name
wallet	String	Ethereum address
email	String	Contact e-mail
approved	Boolean	Approved by admin

CERTIFICATES

FIELD NAME	DATA TYPE	DESCRIPTION
cert_id	String	Primary key (e.g. CC-2026-00125)
inst_id	ObjectId	Foreign key to Institutions
student_name	String	Student name
reg_no	String	Register number
course	String	Course / degree
file_hash	String	SHA-256 hash
ipfs_cid	String	IPFS CID

tx_hash	String	Blockchain transaction
---------	--------	------------------------

VERIFICATIONS

FIELD NAME	DATA TYPE	DESCRIPTION
ver_id	ObjectId	Primary key
cert_id	String	Foreign key to Certificates
result	String	valid / tampered / revoked / not_found
verifier_ip	String	Requester IP
ts	Date	Time

ON-CHAIN RECORD (SOLIDITY STRUCT)

FIELD NAME	DATA TYPE	DESCRIPTION
certId	bytes32	Key - certificate id
fileHash	bytes32	SHA-256 hash
issuer	address	Issuer wallet
ipfsCid	string	IPFS CID
issuedAt	uint256	Block timestamp
revoked	bool	Revocation status

6.6 Algorithm Design

The issue and verify algorithms are based on cryptographic hashing and the immutability of the blockchain.

- 1. Institution logs in and uploads certificate** – The institution's wallet must be in the authorised issuer list.
- 2. Compute SHA-256 hash of the document** – The exact bytes of the PDF are hashed to a 32-byte digest.
- 3. Store hash, issuer ID and timestamp through a smart contract** – `issueCertificate(certId, hash, cid)` is called; the contract rejects duplicates and unauthorised callers.

4. **Generate QR code linked to the record** – The QR code encodes `https://certchain.app/verify/<certId>`.
5. **Verifier scans QR or uploads file** – The certificate ID is obtained from the QR code or entered manually.
6. **Recompute hash and query the contract** – `getCertificate(certId)` is a free read-only call.
7. **Return Valid / Tampered / Not Found** – If no record: Not Found; if revoked: Revoked; if hashes match: Valid; otherwise Tampered.
8. **Maintain audit log of all verifications** – Each verification result is stored off-chain with time.

PSEUDO CODE

```

ISSUE(file, details):
  REQUIRE caller IN authorisedIssuers
  h <- SHA256(file); cid <- IPFS.ADD(file); id <- NEW_CERT_ID()
  CONTRACT.issueCertificate(id, h, cid)          // signed with MetaMask
  RETURN id, QR("https://certchain.app/verify/" + id)

VERIFY(id, file):
  rec <- CONTRACT.getCertificate(id)
  IF rec.issuedAt = 0 THEN result <- NOT_FOUND
  ELSE IF rec.revoked THEN result <- REVOKED
  ELSE IF rec.fileHash = SHA256(file) THEN result <- VALID
  ELSE result <- TAMPERED
  LOG(id, result, now); RETURN result

```

6.7 Modules

The system is divided into modules for the Institution (issuer), the Verifier and the Admin.

MODULES

INSTITUTION

1. LOGIN WITH WALLET
2. ISSUE CERTIFICATE
3. REVOKE CERTIFICATE

VERIFIER

1. SCAN QR CODE
2. UPLOAD AND VERIFY
3. VIEW RESULT

ADMIN

1. APPROVE INSTITUTIONS
2. VIEW AUDIT LOG

INSTITUTION

LOGIN WITH WALLET

The institution staff logs in and connects the MetaMask wallet whose address has been whitelisted by the admin.

ISSUE CERTIFICATE

The staff enters student name, register number, course and year and uploads the certificate PDF. The hash is stored on chain through a MetaMask-signed transaction.

REVOKE CERTIFICATE

If a certificate was issued in error, the issuer revokes it; verification then shows Revoked.

VERIFIER

SCAN QR CODE

The verifier scans the QR code printed on the certificate, which opens the verification page with the certificate ID.

UPLOAD AND VERIFY

The verifier uploads the certificate file. The system recomputes the hash and compares it with the on-chain record.

VIEW RESULT

The result Valid, Tampered, Revoked or Not Found is displayed along with issuer name and issue date.

ADMIN

APPROVE INSTITUTIONS

Admin verifies institutions and adds their wallet addresses to the contract's list of authorised issuers.

VIEW AUDIT LOG

Admin views all issue, revoke and verification events.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

The smart contract was written in Solidity 0.8.20, compiled and tested with Hardhat, and deployed to the Ethereum Sepolia test network through an Alchemy RPC endpoint. The back end is a Node.js Express server that uses Ethers.js, and the front end is a React application with MetaMask integration for signing issue and revoke transactions.

7.2 Implementation

Unit tests were written for the contract in Hardhat (JavaScript) to check that only authorised issuers can issue, that duplicate certificate IDs are rejected, and that revocation works. Tampering was tested by changing a single character in a certificate PDF and confirming that the hash no longer matched.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.

- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	Issue certificate	Valid PDF by authorised issuer	Transaction confirmed, QR generated	Pass
TC02	Unauthorised issuer	Wallet not whitelisted	Transaction reverted 'Not authorised'	Pass
TC03	Duplicate ID	Same certificate ID twice	Reverted 'Already issued'	Pass
TC04	Verify original	Original PDF	VALID with issuer and date	Pass
TC05	Verify edited file	PDF with changed marks	TAMPERED	Pass
TC06	Unknown ID	Random certificate ID	NOT FOUND	Pass
TC07	Revocation	Revoke then verify	REVOKED	Pass
TC08	QR scan	Scan with mobile	Verification page opens with ID	Pass

7.4 Source Code

contracts/CertChain.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract CertChain {
    struct Certificate {
        bytes32 fileHash;
        address issuer;
        string ipfsCid;
        uint256 issuedAt;
        bool revoked;
    }

    address public admin;
    mapping(address => bool) public authorisedIssuers;
    mapping(bytes32 => Certificate) private certificates;

    event CertificateIssued(bytes32 indexed certId, address indexed issuer,
```

```

bytes32 fileHash);
    event CertificateRevoked(bytes32 indexed certId);

    modifier onlyAdmin() { require(msg.sender == admin, "Only admin"); _; }
    modifier onlyIssuer() { require(authorisedIssuers[msg.sender], "Not
authorised"); _; }

    constructor() { admin = msg.sender; }

    function addIssuer(address issuer) external onlyAdmin {
authorisedIssuers[issuer] = true; }
    function removeIssuer(address issuer) external onlyAdmin {
authorisedIssuers[issuer] = false; }

    function issueCertificate(bytes32 certId, bytes32 fileHash, string calldata
cid) external onlyIssuer {
    require(certificates[certId].issuedAt == 0, "Already issued");
    certificates[certId] = Certificate(fileHash, msg.sender, cid,
block.timestamp, false);
    emit CertificateIssued(certId, msg.sender, fileHash);
}

    function revokeCertificate(bytes32 certId) external {
    Certificate storage c = certificates[certId];
    require(c.issuer == msg.sender, "Only original issuer");
    c.revoked = true;
    emit CertificateRevoked(certId);
}

    function getCertificate(bytes32 certId) external view
    returns (bytes32, address, string memory, uint256, bool) {
    Certificate memory c = certificates[certId];
    return (c.fileHash, c.issuer, c.ipfsCid, c.issuedAt, c.revoked);
}
}

```

server/verify.js

```

const express = require("express");
const multer = require("multer");
const crypto = require("crypto");
const { ethers } = require("ethers");
const { MongoClient } = require("mongodb");
const abi = require("../CertChain.json").abi;

const app = express();
const upload = multer({ limits: { fileSize: 5 * 1024 * 1024 } });
const provider = new ethers.JsonRpcProvider(process.env.SEPOLIA_RPC);
const contract = new ethers.Contract(process.env.CONTRACT_ADDRESS, abi,
provider);
let db;
MongoClient.connect(process.env.MONGO_URL).then(c => (db = c.db("certchain")));

const toId = id => ethers.encodeBytes32String(id);

app.post("/api/verify/:certId", upload.single("file"), async (req, res) => {
    const certId = req.params.certId;
    const [fileHash, issuer, cid, issuedAt, revoked] = await
contract.getCertificate(toId(certId));
    let result;
    if (issuedAt === 0n) result = "NOT_FOUND";

```

```

else if (revoked) result = "REVOKED";
else {
  const h = "0x" +
crypto.createHash("sha256").update(req.file.buffer).digest("hex");
  result = h === fileHash ? "VALID" : "TAMPERED";
}
const meta = await db.collection("certificates").findOne({ cert_id: certId });
await db.collection("verifications").insertOne({ cert_id: certId, result, ip:
req.ip, ts: new Date() });
res.json({
  result, issuer, ipfs: cid,
  issuedOn: issuedAt ? new Date(Number(issuedAt) *
1000).toISOString().slice(0, 10) : null,
  student: result === "VALID" ? meta?.student_name : undefined,
  course: result === "VALID" ? meta?.course : undefined,
});
});
});

app.listen(4000, () => console.log("CertChain API on 4000"));

```

7.5 Screen Layout

SCREENSHOTS:

ISSUE CERTIFICATE (INSTITUTION)

The screenshot shows the 'CertChain' web application interface. The top navigation bar is blue with the 'CertChain' logo and links for 'Issue', 'Verify', 'Certificates', 'Audit Log', and 'Institutions'. The main content area is titled 'Issue new certificate' and is divided into two columns.

Left Column (Form Fields):

- Student Name:** A text input field containing 'Priya Ramesh'.
- Register Number:** A text input field containing '21UCS1045'.
- Course:** A text input field containing 'B.Sc. Computer Science - First Class'.
- Certificate PDF:** A dashed box containing a file upload icon and the text 'degree_21UCS1045.pdf (412 KB)'.
- Button:** A blue button labeled 'Hash and Issue on Blockchain'.

Right Column (Wallet Information):

- Wallet:** A section header.
- Connected:** 0x7aC3...91fE (Authorised issuer)
- Network:** Sepolia testnet
- Estimated gas:** 0.00021 ETH

ISSUE SUCCESS

https://certchain.app/

CertChain

[Issue](#)[Verify](#)[Certificates](#)[Audit Log](#)[Institutions](#)

Certificate issued

Issued on blockchain ✓

Certificate ID	CC-2026-00125
SHA-256	9f2c4e1b...a7d03e55
IPFS CID	bafybeigd...k3x4q
Transaction	0x4be1...0c9a (block 6,241,887)
Issued on	14-10-2026 11:42

QR code generated - download and print on the certificate.

Share link: certchain.app/verify/CC-2026-00125

VERIFICATION - VALID

https://certchain.app/

CertChain

[Issue](#)[Verify](#)[Certificates](#)[Audit Log](#)[Institutions](#)

Verify certificate

VALID - Certificate is genuine

Certificate ID	CC-2026-00125
Student	Priya Ramesh
Course	B.Sc. Computer Science
Issued by	Government Arts College (0x7aC3...91fE)
Issued on	14-10-2026

Hash of uploaded file matches the blockchain record.

VERIFICATION - TAMPERED

https://certchain.app/

CertChain

IssueVerifyCertificatesAudit LogInstitutions

Verify certificate

TAMPERED - Document has been modified

Certificate ID	CC-2026-00125
Uploaded file hash	1d77b0a2...c4e910f8
Blockchain hash	9f2c4e1b...a7d03e55
Result	Hashes do not match

Contact the issuing institution. This attempt has been logged.

AUDIT LOG

https://certchain.app/

CertChain

IssueVerifyCertificatesAudit LogInstitutions

Verification audit log

Institution: AllResult: AllLast 30 days

Time	Certificate	Result	Verifier
14-10 12:05	CC-2026-00125	VALID	hr@infotech.com
14-10 12:09	CC-2026-00125	TAMPERED	203.0.113.24
13-10 16:40	CC-2026-00098	VALID	admissions@univ.edu
13-10 10:02	CC-2025-01777	NOT FOUND	198.51.100.7
12-10 09:15	CC-2026-00042	REVOKED	hr@softcorp.in

CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The Blockchain-Based Academic Certificate Verification System provides a tamper-proof method of issuing and verifying academic certificates. By storing SHA-256 hashes in an Ethereum smart contract, any modification of a certificate, however small, is detected instantly. Verification is free, fast and available to anyone through a QR code.

The hybrid design keeps only essential data on chain, which reduces cost and protects privacy, while IPFS and MongoDB handle file storage and audit logs. The project demonstrates a practical solution to certificate fraud for colleges, students and recruiters.

8.2 Future Enhancement

- Batch issuance using Merkle trees to reduce gas cost.
- W3C Verifiable Credentials and DigiLocker integration.
- Mobile wallet app for students to hold their credentials.
- Deployment on a permissioned consortium blockchain of universities.
- Support for mark sheets, transcripts and internship certificates.

CHAPTER IX

BIBLIOGRAPHY

1. Nakamoto, S., "Bitcoin: A Peer-to-Peer Electronic Cash System", 2008.
2. Buterin, V., "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform", White Paper, 2014.
3. Grech, A., and Camilleri, A. F., "Blockchain in Education", JRC Science for Policy Report, Publications Office of the European Union, 2017.
4. Benet, J., "IPFS - Content Addressed, Versioned, P2P File System", arXiv:1407.3561, 2014.
5. National Institute of Standards and Technology, "FIPS PUB 180-4: Secure Hash Standard (SHS)", 2015.
6. Antonopoulos, A. M., and Wood, G., Mastering Ethereum, O'Reilly Media, 2018.

WEBSITE REFERENCE

1. <https://docs.soliditylang.org>
2. <https://hardhat.org>
3. <https://docs.ethers.org>
4. <https://www.blockcerts.org>